



Università
di Catania

Dipartimento di
Ingegneria Elettrica Elettronica e
Informatica

Advanced Programming Languages

C++: namespaces, dynamic memory, casting & templates

Docente:

Marco Grassia

marco.grassia@unict.it

A.A. 2025/2026

Namespaces

- A ***namespace*** is a **declarative region** that **provides a scope to the identifiers inside it**
- Namespaces are **used to organize code into logical groups and to prevent name collisions** that can occur especially when your code base includes multiple libraries
- **All identifiers at namespace scope are visible to one another without qualification**
- **Outside** the namespace, its **members can be accessed using the fully qualified name** for each identifier in the form:

namespace_name::**namespace_member**

```
namespace circle {  
    const double pi = 3.14159;  
    double circle_area(double radius) {  
        return pi * radius * radius;  
    }  
}  
double area = circle::area(5.0);
```

Namespaces

- An **alias** for a namespace name **can be created using**
`namespace ns = namespace_name;`
- The namespace can be used then with the alias:
`ns::namespace_member`
- User-defined libraries should live inside namespaces

```
namespace circle {  
    const double pi = 3.14159;  
    double circle_area(double radius) {  
        return pi * radius * radius;  
    }  
}  
  
double area = circle::area(5.0);  
  
namespace c = circle;  
  
area = c::area(5.0);  
double pi = c::pi;
```

using keyword

- *using* is quite a **versatile keyword** in C++
- It can work as:
 - **Type alias declaration** (C++11)
 - A **declaration**: imports a name from a namespace
 - A **directive** imports *all* names from a namespace
 - A **directive for *enums***: to import enumerators (C++20)

using directive

- ***using*** can be used as a **directive to import all names from a namespace in the current scope**

`using namespace namespace_name;`

- **Doing so, especially at global scope, is a bad practice. If needed, limit to a small local scope and never in headers (affects all includers → lookup pollution)**
- In fact, it is **risky**: namespace changes (e.g., new versions) or just “importing” multiple ones **can cause collisions**

```
#include <iostream>

using namespace std;

int main() {
    cout << "Hello, World!" << endl;

    cout << "Welcome" << "to C++";

    return 0;
}
```

using declaration

- ***using* declaration to bring just the specified name(s)** from a namespace in the current scope
`using namespace::member [, ...];`
- **Safer** than importing everything, **should still be used in small scopes**
- Affects **unqualified lookup** only in the scope where it appears
- **Can be used to disambiguate** when multiple libraries define the same name

```
namespace math {  
    double sqr(double x);  
    double pow(double x, int n);  
}
```

```
// bring only 'sqr'  
using math::sqr;  
double y = sqr(3.0);
```

```
// bring both 'sqr' and 'pow'  
using math::sqr, math::pow;  
double z = pow(2.0, 3);
```

using declaration

- ***using* declaration to bring just the specified name(s)** from a namespace in the current scope
`using namespace::member [, ...];`
- **Safer** than importing everything, **should still be used in small scopes**
- Affects **unqualified lookup** only in the scope where it appears
- **Can be used to disambiguate** when multiple libraries define the same name

```
#include <iostream>
```

```
using std::cout, std::endl;
```

```
int main() {  
    cout << "Hello, World!" << endl;  
  
    cout << "Welcome" << "to C++";  
  
    return 0;  
}
```

using declaration and overloads

- **Interactions with ADL & Overload Resolution**
- using namespace can **inject** candidate functions that change which overload gets picked
- using-declaration composes overload sets **intentionally**:
- Be careful with **ADL + directives**: prefer explicit qualification or targeted using

```
namespace A { void f(int); }  
namespace B { void f(double); }
```

```
using A::f;  
using B::f;
```

```
// now both overloads visible with an additional using  
// f(int) and f(double) form one set in this scope
```


Nested namespaces

- Namespaces **can also be nested**
- Since C++17, namespaces definition can be concatenated
 - Instead of :
 - ```
namespace C {
 namespace D {
 void k();
 }
}
```
    - ```
namespace C::D {  
    void k();  
}
```

```
namespace A {  
    const double e = 2.71828;  
  
    namespace B {  
        const double pi = 3.14159;  
  
        void f();  
    }  
}
```

```
// Since C++17:  
namespace A::B {  
    const double g = 9.81;  
    void h();  
}
```

```
A::e;  
A::B::pi;  
A::B::f();  
A::B::g();
```

Inline namespaces

- **Members of an *inline* namespace are treated as if they are members of the enclosing namespace** in many situations
- **Used for library versioning** and API evolution **without breaking existing code**
 - Allows backward compatibility while introducing new versions
- **Declared with *inline* keyword**
- Older members can be invoked using fully qualified names

```
namespace lib {  
    inline namespace v2 {  
        void f(); // preferred version  
    }  
    namespace v1 {  
        void f(); // older version  
    }  
}  
  
lib::f(); // calls v2::f by default  
lib::v2::f(); // calls v2::f explicitly  
lib::v1::f(); // explicit older version
```

Argument-Dependent Lookup (ADL)

- **When calling an unqualified function, the compiler also searches the namespaces of the argument types**
- **Use fully qualified names to avoid this behavior**

```
namespace math {  
    struct X {};  
    void print(X) { std::cout << "math::print\n"; }  
}  
  
using std::print;  
using math::print;  
  
math::X x;  
print(x); // finds math::print via ADL
```

Namespaces and headers

- To avoid polluting the global namespace (when including the header), **always wrap all declarations in a namespace**

```
// header.hpp
#ifndef MYLIB_HEADER_HPP
#define MYLIB_HEADER_HPP
namespace mylib {
    void greet();
}
#endif // MYLIB_HEADER_HPP

// header.cpp
namespace mylib {
    void greet() {
        std::println("Hello from mylib!");
    }
}
```

Namespaces:

Best practices

- **Do:**

- Use namespaces for libraries and modules
- Use **qualified names** in headers
- Use **inline namespaces** for versioning
- Use **anonymous namespaces** for TU-local entities

- **Avoid:**

- using namespace ...; in headers → pollutes global scope
- Overusing using declarations in large scopes → ambiguity

Exercise:

Matrix namespace

- Use serialization (row-major ordering)

```
namespace Matrix {  
    inline int idx(int r, int c, int C) { return r*C + c; }  
    void set(int* m, int R, int C, int r, int c, int val);  
    int get(const int* m, int R, int C, int r, int c);  
  
    namespace ew {  
        void add(const int* A, const int* B, size_t R, size_t C, int* result);  
    }  
    namespace mx { // matrix-wise operations  
        void matmul(const int* A, const int* B, size_t AR, size_t AC, size_t BC, int* result);  
    }  
}
```

Dynamic memory

Why dynamic memory?

- Static arrays and objects have **fixed size** known at compile time
- Dynamic memory allows:
 - **Variable size** determined at runtime
 - **Lifetime control** beyond local scope
- It is often used for:
 - Large data structures
 - Objects whose size depends on user input or runtime constraints

The old way: C-style *malloc / calloc / realloc / free*

- Declared in `<cstdlib>`
- **Allocates raw memory, not objects in the “new” sense**
 - **No constructors/destructors called**
 - **No type safety**
- **Returns void***: requires **manual cast** in C++
- Error-prone (manual size, cast, and free)

```
#include <cstdlib>
int main() {
    const int N = 100;

    int *array = static_cast<int*>(malloc(N * sizeof(int)));
    int *zero_array = static_cast<int*>(calloc(N, sizeof(int)));

    std::free(array);
    std::free(zero_array);

    return 0;
}
```

The C++ way: Object Instantiation

- Object Instantiation = creating an object + initializing it
- Objects can be instantiated by:
 - Declaring an object (statically)
 - Creating an object (dynamically)

The C++ way:

new / delete

- ***new* allocates and instantiates objects on the free store**
- Syntax:

`new T(args)`
- Allocates raw storage (via `::operator new`), then **constructs** a *T*
- Returns *T**, a pointer of the correct type
- On failure, **throws** `std::bad_alloc` (unless `nothrow` form used)
- *new T*; for scalars allocates with indeterminate values (default init)

// default-initialization

`T* p1 = new T;`

// value-initialization

`T* p2 = new T();`

// list-initialization (C++11+)

`T* p3 = new T{args};`

The C++ way:

new / delete

- **delete** calls the destructor (if any) and frees storage the object
- Syntax:

`delete p`
- `delete nullptr;` is **safe** (no-op)
- **Double delete** or deleting a **non-owned pointer** $\Rightarrow$ undefined behavior
- **Memory leak** if *delete* is not called
- **Dangling pointer**: pointer/reference to a **destroyed** object. Set removed object pointers to *nullptr*

```
T* p1 = new T;
```

```
delete p1;
```

```
p1 = nullptr;
```

The C++ way: *new[] / delete[]*

- ***new* and *delete* can also construct and destruct dynamic arrays**
- The constructor (and destructor) is invoked per element
- Syntax:

`new T[N]`

`delete[] p`

- **Must match:** `new` → `delete`, `new[]` → `delete[]`

```
T* a1 = new T[N];
```

```
int* a2 = new int[N]{0};
```

```
int* a3 = new int[N]{3};
```

```
int* a4 = new int[N](3);
```

```
// array delete → N elements destructed
```

```
delete [] a1;
```

```
delete [] a2;
```

```
delete [] a3;
```

```
delete [] a4;
```

Dynamic memory examples

```
int *p = nullptr; // null pointer
*p = 7;           // Error: dereferencing null pointer
p = new int;      // Dynamic allocation of an integer
*p = 7;           // OK, pointer now points to valid memory
delete p;         // Do not forget to deallocate to avoid memory leaks
*p = 7;           // Error: p is now a dangling pointer
```

```
double *v = nullptr; // null pointer,
v = new double[1024]; // Dynamic allocation of an array of 1024 doubles
v[907] = 3.14;        // OK, pointer points to valid memory
```

```
delete[] v;         // Deallocate array memory (note the use of delete[])
v[907] = 2.71;       // Error: v is now a dangling pointer
```

placement new: construct *into* existing storage

- Placement *new* **allows passing an address to construct an object at a specific memory location**
- The **memory must be already allocated**, as it **does not allocate**
- **Syntax:**
`new (void* addr) T(args...)`
- Advanced feature: misuse leads to hard Undefined Behavior

```
char buffer[sizeof(int)];  
int* p = new (buffer) int(42); // placement new
```

```
#include <new> // std::aligned_storage or std::align if needed  
  
alignas(double) unsigned char buf[sizeof(double)];  
double* pd = new (buf) double(3.14); // construct in-place  
// use *pd ...  
pd->~double(); // explicit destructor call  
// buf is just raw storage; no delete here
```

placement new: construct *into* existing storage

- **You must ensure:**
 - The storage is suitably **aligned** and **sized** for T
 - You later call the **destructor explicitly** and then free the storage appropriately (if you own it)
- Use it for **custom allocators, memory pools, or in-place transforms; e.g., in embedded systems**

```
char buffer[sizeof(int)];  
int* p = new (buffer) int(42); // placement new
```

```
#include <new> // std::aligned_storage or std::align if needed  
  
alignas(double) unsigned char buf[sizeof(double)];  
double* pd = new (buf) double(3.14); // construct in-place  
// use *pd ...  
pd->~double(); // explicit destructor call  
// buf is just raw storage; no delete here
```


Esempio di Memory Leak

si alloca memoria per
un *double*

ora *p_x* punta all'area di
memoria contenente *x*
ma non si è liberata
quella precedentemente allocata!!!

cancelliamo la memoria
precedentemente allocata
per il *double*

```
#include <iostream.h>

using namespace std;

int main()
{
    while(1) {
        double x = 20;

        double * p_x = new double;

        delete p_x;
        p_x = &x; // Attenzione!! Memory leak!!

        cout << *p_x << endl;
    }

    return 0;
}
```

Spot the bug

```
int* p = new int(5);  
auto f = [p] { return *p; };  
delete p;
```

```
int x = f();
```

Exercise

- Define a namespace for operations on array of integers
 - Dynamic allocation and initialization to a given value (default = 0)
 - Min
 - Max
 - Resize: `bool resize_array(int*& a, int& n, int new_n);`
 - Deep copy: `int* deep_copy(const int* src, int n);`
 - For each (copy paste from before?)
 - Find first occurrence

Exercise:

Matrix namespace

- Use serialization (row-major ordering)
- Define the IntMatrix type (simple structure with data and sizes)
- Declare a pointer to the type
- Implement the following functions
 - `inline int idx(const IntMatrix&m, int r, int c);`
 - `void set(IntMatrix& m, int r, int c, int val);`
 - `int get(const IntMatrix& m, int r, int c);`
- Subnamespaces:

```
namespace dot {  
    IntMatrix multiply(const IntMatrix& a, const IntMatrix& b);  
}
```

```
namespace matmul {  
    IntMatrix multiply(const IntMatrix& a, const IntMatrix& b);  
}
```

Type casting

APL: C++

Type casting

- C++ is **strongly typed**: implicit conversions happen only when safe (e.g., widening from short to int)
- When you need **to override type rules explicitly**, you must use **explicit casting**
- Casting is about **telling the compiler**: “I know what I’m doing. Convert this type.”
- **Casts are widely (mis)used**: modern C++ has rules and constructs that eliminate the need for casts in many contexts
- **Casts are a well-known source of errors and make some optimizations unreliable**

C-style casts

- C++ allows for C-style type casting with the syntax:
(`new_type`) expression
- **Avoid this kind of cast:** it tries the various C++-style casts in order, until it finds one that works
 - It **can remove const or reinterpret bits**;
 - It **allows dangerous conversions**, while **suppressing** any **warnings or errors** from the compiler, and **with** complete **disregard for the C++ type system**
 - **Ambiguous about intent:** it does not clearly show the programmer's intent, and are hard to search for in a code base
- Functional casts `T(expression)` can behave similarly

C-style cast: issues

- They allow you to **perform dubious casts between integers and pointers**
- They allow you to **cast away const** and volatile qualifiers
- They allow you to **cast between pointers to unrelated types**

```
const char *message = "12345";  
unsigned long bad = (unsigned long)message;  
// accidental pointer-to-int conversion  
long also_bad = long(message); // functional cast, bad as well
```

```
const char *message = "12345";  
char *very_bad = (char *)message; // casts away const
```

```
struct Pineapple { /* ... */ };  
class Bulldozer { /* ... */ };  
  
Pineapple p;  
Bulldozer *b = (Bulldozer *)&p; // b points to a pineapple, not a bulldozer
```


Avoid casting at all

1. Sometimes you don't need an explicit cast.
Just let the type system do its thing
2. **Rely on the well-defined, safe, implicit conversions** between pointer (or reference) types

In the case of literals
you can use a literal suffix to avoid a cast:

```
float f_bad = (float) 3;  
float f = 3.14f;
```

```
struct Base {};  
struct Derived : Base {};
```

```
Derived d;  
Base* pb = &d; // safe, implicit
```

```
Base b;  
Derived *pd = &b; // unsafe, implicit
```

```
int* p = nullptr;  
const int* cp = p; // safe  
int* q = cp; // unsafe, removes const, not allowed
```

```
void* p = &b; // void* can point to any type  
int* pb2 = p; // unsafe pointer cast, not allowed
```

Avoid casting at all

$T\{e\}$ is the only safe and general expression for constructing a value of type T from an expression e . The casts notations $T(e)$ and $(T)e$ are neither safe nor general.

3. Use $T\{e\}$ construction syntax (curly braces) instead of using parentheses

- This makes it explicit that construction is desired
- **If conversion is valid, the effect is the same as a C-style cast; if invalid or narrowing type conversions are rightly rejected**

```
int integer_value = 100;
```

```
long long_value = long{integer_value}; // safe conversion
```

```
double d1 = double{integer_value}; // ERROR: narrowing
```

```
double d2 = float{3.14f}; // OK, from float to double
```

Avoid casting at all

$T\{e\}$ is the only safe and general expression for constructing a value of type T from an expression e . The casts notations $T(e)$ and $(T)e$ are neither safe nor general.

3. Use $T\{e\}$ construction syntax

```
void use(char ch, int i, double d, char* p, long long lng) {  
    int x1 = int{ch};    // OK, but redundant  
    int x2 = int{d};    // error: double->int narrowing; use a cast if you need to  
    int x3 = int{p};    // error: pointer to->int; use a reinterpret_cast if you really need to  
    int x4 = int{lng};  // error: long long->int narrowing; use a cast if you need to  
  
    int y1 = int(ch);    // OK, but redundant  
    int y2 = int(d);    // bad: double->int narrowing; use a cast if you need to  
    int y3 = int(p);    // bad: pointer to->int; use a reinterpret_cast if you really need to  
    int y4 = int(lng);   // bad: long long->int narrowing; use a cast if you need to  
  
    int z1 = (int)ch;    // OK, but redundant  
    int z2 = (int)d;    // bad: double->int narrowing; use a cast if you need to  
    int z3 = (int)p;    // bad: pointer to->int; use a reinterpret_cast if you really need to  
    int z4 = (int)lng;   // bad: long long->int narrowing; use a cast if you need to  
}
```

C++ named casts

- **If you must** use a cast, **use a named cast**: they are **more specific** than a C-style or functional cast, **allowing the compiler to catch some errors**
- Moreover, they **improve readability**, making the intent clear
- (some of) the named casts are:
 - *static_cast*
 - *const_cast*
 - *reinterpret_cast*
 - *dynamic_cast*
 - *gsl::narrow*
 - *gsl::narrow_cast*

C++ named casts:

`static_cast<T>(expr)`

- **Converts types with compile-time checks** between related types. Used for:
 - **Numeric conversions** (e.g., double → int) (against ISO C++ guidelines)
 - **Up/down casts** in inheritance, safe if you know the hierarchy
 - **Convert from `void*`**
- It **disallows casts that would violate the rules of the type system, such as casting away qualifiers or converting between unrelated types**

```
short from_long = static_cast<int>(100LL);
```

```
const char *message = "12345";  
// compile-time error (as it should)  
char *little_better = static_cast<char *>(message);
```

```
Pineapple p;  
// compile-time error (as it should)  
Bulldozer *b = static_cast<Bulldozer *>(&p);
```

C++ named casts:

`const_cast<T>(expr)`

- **Changes** only cv-qualifiers (**const/volatile**) of **pointers and references**. It does **not** **change the underlying type**
- Works on pointers, references, and the implicit this in member functions
- **Warning: modifying truly const objects is undefined behavior**

```
const int x = 42;  
int* p = const_cast<int*>(&x);  
*p = 99; // UB if x was originally const  
std::print("Modified x: {}\n", x);
```

```
const char* str = "Hello, World!";  
char* modifiable_str = const_cast<char*>(str);  
modifiable_str[0] = 'h'; // UB: modifying string literal  
std::print("Modified string: {}\n", modifiable_str);
```

C++ named casts:

`dynamic_cast<T>(expr)`

- **Performs safe downcast (with runtime type check) in polymorphic hierarchies** (requires at least one virtual function)
- **Returns *nullptr* for invalid pointer casts or throws `std::bad_cast` for references** (as they cannot be null)
- **Uses RTTI** (RunTime Type Information or RunTime Type Identification) **lookup**, avoid in performance-critical code
- We'll see some examples when discussing about Classes

C++ named casts:

`reinterpret_cast<T>(expr)`

- It is primarily a **compile-time directive** which **instructs the compiler to treat expression as if it had the type target-type: low-level, bitwise reinterpretation of types**
- **No check is performed at all** (neither compile-time or run-time check): programming **errors can go undetected** at development time **and cause subtle or catastrophic errors** in your program's behavior
- **Use only for low-level tasks** (e.g., serialization, hardware access)

Guidelines Support Library (GSL)

- The **Guidelines Support Library** provides **utilities recommended by the C++ Core Guidelines**
- Includes **safer alternatives for common operations** (e.g., bounds checking, type conversions)
- One implementation is Microsoft's <https://github.com/microsoft/GSL>

C++ named casts:

`gsl::narrow_cast<T>(expr)`

- Don't use a `static_cast` for arithmetic conversions but brace initialization (`{ }`), `gsl::narrow_cast`, or `gsl::narrow`
- `gsl::narrow_cast<T>(expr)` **behaves like** `static_cast<T>(expr)` **but states clearly that you're willing to use a narrowing conversion and lose data**

```
int a = gsl::narrow_cast<int>(3.14);
std::print("Narrow cast result: {}\n", a);

try {
    int b = gsl::narrow<int>(3.14);
    std::print("Narrow result: {}\n", b);
}
catch (const gsl::narrowing_error& e) {
    std::print("Narrowing error: {}\n", e.what());
}
```

C++ named casts:

`gsl::narrow<T>(expr);`

- `gsl::narrow<T>(x)` is `static_cast<T>(x)` if `static_cast<T>(x) == x` or it throws
- `gsl::narrow<T>(expr)` **behaves like** `static_cast<T>(expr)` **only if no information loss** is going to happen
- It will throw an exception (`gsl::narrowing_error`) otherwise

```
int a = gsl::narrow_cast<int>(3.14);
std::print("Narrow cast result: {}\n", a);

try {
    int b = gsl::narrow<int>(3.14);
    std::print("Narrow result: {}\n", b);
}
catch (const gsl::narrowing_error& e) {
    std::print("Narrowing error: {}\n", e.what());
}
```

Templates

Advanced Programming Languages: C++

C++ templates:

why do we need templates?

- Suppose you **need** a `max()` function for **different types**, you **end up writing multiple versions of the same logic**
- Overloading helps, but becomes **tedious and error-prone**, casting is not an option as custom types require manual overloads
- **Large projects** with lots of types **amplify the issue**
- This leads to **code duplication, inconsistency, and maintenance nightmares**, with projects scaling poorly

```
int max(int a, int b);
```

```
float max(float a, float b);
```

```
char max(char a, char b);
```

```
double max(double a, double b);
```

```
std::string max(std::string a, std::string b);
```

C++ templates: generic programming

- **Templates enable generic programming** by allowing functions and classes to operate with generic types
- The type becomes a “parameter”
- Templates were **introduced to improve code reuse, type safety,** and performance w.r.t. macros
- Two main types:
 - **Function Templates**
 - **Class Templates**

template keyword

type parameter

```
template<typename T>  
T max(T a, T b);
```

Header

```
template<typename T>  
T max(T a, T b) {  
    return (a > b) ? a : b;  
}
```

C++ templates: syntax

- The syntax `template<typename T>` declares a **type parameter T** that can be used
- **The compiler generates a separate function for each T, generating type-specific code**
- This means that defining a template function generates not single functions but a family of functions (**not compiled** until they are **instantiated**)
- Templates requires **full definition to be visible at point of use** (typically in headers)

```
template<typename T>
T max(T a, T b);
```

```
template<typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}
```

```
// Type parameter provided
max<double>(4.2, 5.5);
max<int>(2, 3);
```

```
// Type deduction
max(2, 3);    // T is int
max(4.2, 5.5); // T is double
max('a', 'b'); // T is char
```

C++ templates: template selection

- When calling a template like `max<double>(3.5, 7.2)`, the compiler:
 - **Extracts the type** used in the call (double)
 - **Generates a concrete function** (e.g., `max_double`) from the template
 - Uses **placeholder names** (e.g., `T`) in the template definition
 - Transforms the template call into a **real function call**

```
template<typename T>  
T max(T a, T b);
```

```
template<typename T>  
T max(T a, T b) {  
    return (a > b) ? a : b;  
}
```

```
max<double>(3.5, 7.2);
```


C++ templates: template selection

- The compiler instantiates (compiles) only **matching templates, once per unique type combination**
 - `max<int>(...)` → `max_int`
 - `max<double>(...)` → `max_double`
- This ensures **efficiency** and **type safety**
- However, **with respect to Java generics, this leads to:**
 - **Code bloat** (multiple instantiations)
 - **Better performance** (no runtime overhead)

```
template<typename T>  
T max(T a, T b);
```

```
template<typename T>  
T max(T a, T b) {  
    return (a > b) ? a : b;  
}
```

```
// Type parameter provided  
max<double>(4.2, 5.5);  
max<int>(2, 3);
```

```
// Type deduction  
max(2, 3);    // T is int  
max(4.2, 5.5); // T is double  
max('a', 'b'); // T is char
```

C++ templates: type deduction

- To instantiate a template, **every** template **argument must be known**, but not every template argument has to be specified
- Templates support **type deduction**, based on **function arguments**, not return type
- **Template selection** is performed using **both type deduction and overload resolution**

```
// Type parameter provided  
max<double>(4.2, 5.5); // T is double  
max<int>(2, 3); // T is int  
  
// Type deduction  
max(2, 3); // T inferred as int  
max(4.2, 5.5); // T inferred as double  
max('a', 'b'); // T inferred as char
```

C++ templates: type deduction and template selection

- **Type deduction** is part of the **overload resolution process**
- If the compiler **cannot deduce** the template parameters, the candidate is **discarded**
- Deduction must succeed for a template to be considered
- If a **non-template overload** matches exactly, it is preferred over a template (unless the template argument is provided)

```
template<typename T>  
void f(T a, T b);
```

```
// Deduction fails: T cannot be both int and double  
f(1, 2.0);
```

```
void max(int, int);  
template<typename T>  
T max(T, T);
```

```
// Calls non-template version  
max(3, 4);  
// Calls to template version:  
max<int>(3, 4); // since the <int> forces T to be int  
  
max<double>(3.0, 4.0);
```

C++ templates: type deduction and template selection

- When a function call is made:
 1. The compiler collects **all viable overloads**
 2. For each template candidate, it attempts **type deduction (if type is not explicitly provided) to match types**. If deduction succeeds, the candidate is considered
 3. The **best match is selected via overload resolution**. Note that the **compiler prefers non-template functions** if they match exactly

```
template<typename T>  
T max(T a, T b);
```

```
template<typename T>  
T max(T a, T b) {  
    return (a > b) ? a : b;  
}
```

```
// Type parameter provided  
max<double>(4.2, 5.5);  
max<int>(2, 3);
```

```
// Type deduction  
max(2, 3);    // T is int  
max(4.2, 5.5); // T is double  
max('a', 'b'); // T is char
```

C++ templates: best practices

- Keep **templates** (definitions) **in headers**
- **Avoid** code bloat with **excessive instantiations**
- **Specialize carefully**
- ~~Use concepts (C++20) to improve readability~~

```
template<typename T>  
T max(T a, T b);  
  
template<typename T>  
T max(T a, T b) {  
    return (a > b) ? a : b;  
}
```

Type Deduction vs Type inference

Feature	Type Deduction	Type Inference
Context	Template instantiation	Variable declaration
Trigger	Function/class template call	auto, decltype, structured bindings
Target	Template parameters	Variable types
Example Keyword	template<typename T>	auto, decltype
C++ Version	Since C++98	Since C++11

C++ templates: why not just macros?

In C, there is another mechanism to define “generic” functions: pre-processor macros

Aspect	Macros (#define)	Templates
Language Level	Preprocessor feature; not part of C++ type system	Fully integrated into the C++ language and type system
Semantics	Pure text substitution before compilation	Type-aware, checked during compilation
Syntax	Function-like macro syntax (no type info)	Function or class template syntax (type parameters explicit)
Type Checking	None: compiler sees only substituted text	Full compile-time type checking
Error Detection	Errors appear after substitution , often cryptic	Clear , type-safe errors at compile time
Implementation	No parsing or semantic analysis; prone to subtle bugs	Parsed and compiled like normal code ; safe and maintainable
Runtime Cost	No overhead (inline by nature)	Typically no overhead if inlined ; same as macro in optimized builds
Debugging	Hard: debugger shows expanded code, not macro call	Easy: debugger shows actual function calls and types
Portability	Risky: text substitution can break across platforms	Portable: templates follow standard C++ rules

C++ templates: why not just macros? An example

- Suppose to define a Max macro
- Suppose to invoke it
- It would become, after the pre-processor substitution
- Or suppose to have this is meaningless and yet compiles

```
#define Max(a,b) ((a)>(b)?(a):(b))
```

```
Max(x++, y--)
```

```
((x++)>(y--)?(x++):(y--)) // side effects!
```

```
char* p;  
Max(p, 1000)
```


C++ templates: why not just macros? An example

Example

```
char* p;  
Max (p, 1000)
```

```
Max (++x, --y)
```

Macro Behavior

Compiles, but result is
meaningless; no type safety

Both `x` and `y` incremented
twice (due to text
substitution)

Template Behavior

Compilation error: type
mismatch detected

Each argument evaluated
once (function semantics)

C++ templates: comparison

- Compared to generics, templates offer **more flexibility** and **compile-time power**, at the cost of **complexity**

Feature	C++ Templates	Java/C# Generics
Instantiation	Compile-time (code generation)	Runtime (type erasure)
Performance	High (no overhead)	Lower (due to type erasure)
Type Safety	Strong	Strong
Specialization	Supported	Not supported
Reflection	Not available	Available
Constraints	Via Concepts (C++20)	Via bounded types